ShipPulse
Solution Guide

Version 2.0 | Date: March 02, 2026

How to Use This Solution Guide

This guide gives exact steps, why we do them, common mistakes, and troubleshooting. It matches the Student Assignment tasks in the same order.

Golden Rules (Real-World + Production-Grade)

- Use PRs + CI checks before merging (avoid direct pushes to protected branches).
- Deploy Dev automatically, deploy Prod with stronger controls (approval/branch restrictions).
- Use IaC for repeatable infrastructure; portal only for learning tasks that are explicitly requested.
- Separate deployment identity (GitHub -> Azure) from runtime identity (Web App -> Key Vault).
- Add monitoring early; deployment without visibility is incomplete.

Preflight: Cloud Shell + Repo

Cloud Shell check

Cloud Shell includes Azure CLI and Bicep, so no local install is required.

Run:

```
az version
az bicep version
```

If Azure CLI is not available, re-open Cloud Shell or choose Bash.

Repo push (starter code)

Expected branches: develop and main. Workflow files exist under .github/workflows/.

Day 2: Deploy Dev Infra via Bicep (Cloud Shell)

Step 1: Create resource groups

```
az group create -n rg-shippulse-dev -l centralindia
az group create -n rg-shippulse-prod -l centralindia
```

Step 2: Set unique suffix and Key Vault naming

Key Vault name must be globally unique and only letters/numbers. Use a short suffix like demo01.

Example: kvshippulsedevDemo01 (no hyphens).

Step 3: Generate SSH key (Cloud Shell)

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/shippulse_id_rsa -N ""
cat ~/.ssh/shippulse_id_rsa.pub
```

Copy the public key into infra/env/dev.bicepparam -> jumpboxSshPublicKey.

Download the private key to your laptop (Cloud Shell file download) and keep it safe. You will SSH using it.

Step 4: Set SSH allowed CIDR

Find your public IP and set sshAllowedCidr = <ip>/32 in dev.bicepparam.

Why: startup-grade security - allow SSH only from your IP.

Step 5: Deploy

```
az deployment group validate -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group what-if  -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
az deployment group create   -g rg-shippulse-dev -f infra/main.bicep -p
infra/env/dev.bicepparam
```

Why we do validate + what-if

It catches parameter mistakes early and shows what Azure will change before executing the deployment. This reduces accidental deletes/overwrites in real teams.

Day 2: VM + Nginx

SSH into VM

Use the public IP output from deployment, then:

```
ssh -i <PRIVATE_KEY_PATH> azureuser@<JUMPBOX_PUBLIC_IP>
```

Install Nginx

```
sudo apt-get update
sudo apt-get install -y nginx
sudo systemctl enable --now nginx
curl -I http://localhost
```

Common issues + fixes

• Cannot SSH: check NSG rule source is your exact public IP/32; ensure you updated sshAllowedCidr.

• Permission denied (publickey): ensure you used the correct private key matching the public key in bicepparam.

• Port 80 not reachable: check NSG has Allow-HTTP-80 and VM is running.

Day 3: Deploy Backend to App Service (GitHub Actions)

Option A (beginner): Service Principal secret

You already learned this. Use it first to get the pipeline working quickly.

Create SP in Cloud Shell:

```
az ad sp create-for-rbac --name sp-shippulse-cicd --role Contributor --scopes /subscriptions/<SUB_ID>
```

Save clientId, tenantId, subscriptionId, and clientSecret in GitHub environment secrets.

Option B (production-grade): OIDC (recommended upgrade)

OIDC avoids long-lived secrets. Use it as a Day 7 upgrade.

Reference: https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect

Required GitHub secrets (dev environment)

- AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID
- AZURE_RG_DEV = rg-shippulse-dev
- AZURE_WEBAPP_NAME_DEV = app-shippulse-api-dev-<suffix>

Deploy

Merge to develop, then confirm workflow 'Deploy Backend (App Service)' runs.

Validation: open https://<dev-webapp>.azurewebsites.net/health

Why this pipeline is production-grade

It builds and tests on the runner, publishes an artifact, deploys infrastructure via IaC, and then deploys the exact built output to App Service. This reduces 'works on my machine' drift.

Day 4: Deploy Frontend to Static Web Apps

Create SWA via Portal

We use Portal here because it reliably wires GitHub permissions and can auto-create a workflow. This avoids beginner setup friction for the token. Later, you can migrate SWA provisioning into IaC if needed.

Configure token + API base URL

Set GitHub secrets:

- AZURE_STATIC_WEB_APPS_API_TOKEN_DEV
- API_BASE_URL_DEV = https://<dev-webapp>.azurewebsites.net
- AZURE_STATIC_WEB_APPS_API_TOKEN_PROD
- API_BASE_URL_PROD = https://<prod-webapp>.azurewebsites.net

How the frontend build works without Node

Frontend is Blazor WebAssembly. It builds using dotnet publish. The workflow writes wwwroot/appsettings.json from the template and injects the API URL.

This fits locked-down laptops because builds happen in GitHub Actions, not locally.

Day 5: Key Vault + Managed Identity (Runtime Secrets)

Create secret

Portal -> Key Vault -> Secrets -> Generate/Import:

Name: ExternalApi--ApiKey | Value: DEMO-123

Grant access (RBAC)

Portal -> Key Vault -> Access control (IAM) -> Add role assignment:

- Role: Key Vault Secrets User
- Assign to: Managed identity
- Select: the Web App (dev first, then prod)

Why managed identity matters

No credentials are stored in code or config. Azure issues identity tokens to the Web App automatically. This reduces secret leakage risk and simplifies rotation.

Common issues

- Key Vault reference shows 'Access denied': RBAC role not assigned or not propagated yet. Wait a few minutes and restart app.
- Key Vault name invalid: must be globally unique, 3-24 chars, letters/numbers only (avoid hyphens). Update bicepparam.

Day 6: Monitoring + Alerts

Enable Application Insights

Portal -> Web App -> Application Insights -> ensure connected to the created resource.

Generate traffic and confirm telemetry (Requests).

Create alert

Create an alert rule on the prod Web App (5xx or failed requests).

Why: even startups need a basic signal when production is broken.

Day 6 Optional: Self-hosted runner on Jumpbox

Why: Learn tradeoffs. Self-hosted runners give more control, but you maintain OS patches, uptime, and security.

Steps:

- GitHub Repo -> Settings -> Actions -> Runners -> New self-hosted runner
- SSH to VM and follow the Ubuntu commands shown by GitHub
- Create a test workflow job with runs-on: self-hosted

Troubleshooting:

- Runner offline: ensure the runner service is running; check VM is started; avoid reboot without service enable.
- Job stuck: label mismatch; use runs-on: [self-hosted] and match labels if added.

Day 7: Production Polish + OIDC Upgrade

Hardening checklist (why)

• HTTPS only and TLS 1.2 minimum (reduce insecure traffic).

• Prod deploy restricted to main (reduce accidental releases).

• Approval gate for prod environment (human check).

• Rollback notes (restore last good version quickly).

OIDC upgrade steps (high-level)

Follow Microsoft OIDC guide. Create an Entra app registration, add a Federated Credential for your GitHub repo, and assign RBAC.

Reference: https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect

Interview-ready talking points

• I used Cloud Shell because laptops were locked down; Azure operations still became repeatable via IaC and pipelines.

• I built Dev first for speed, then added Prod gates for safety.

• I separated deployment identity from runtime identity (Key Vault + managed identity).

• I enabled monitoring and created a basic alert to make the system operable.

• I compared GitHub-hosted vs self-hosted runners and explained the tradeoffs.

References

• Cloud Shell tools: https://learn.microsoft.com/en-us/azure/cloud-shell/features

• Deploy App Service with GitHub Actions: https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions

• GitHub Environments: https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments

• Key Vault references: https://learn.microsoft.com/en-us/azure/app-service/app-service-key-vault-references

• Codeless App Insights: https://learn.microsoft.com/en-us/azure/azure-monitor/app/codeless-app-service